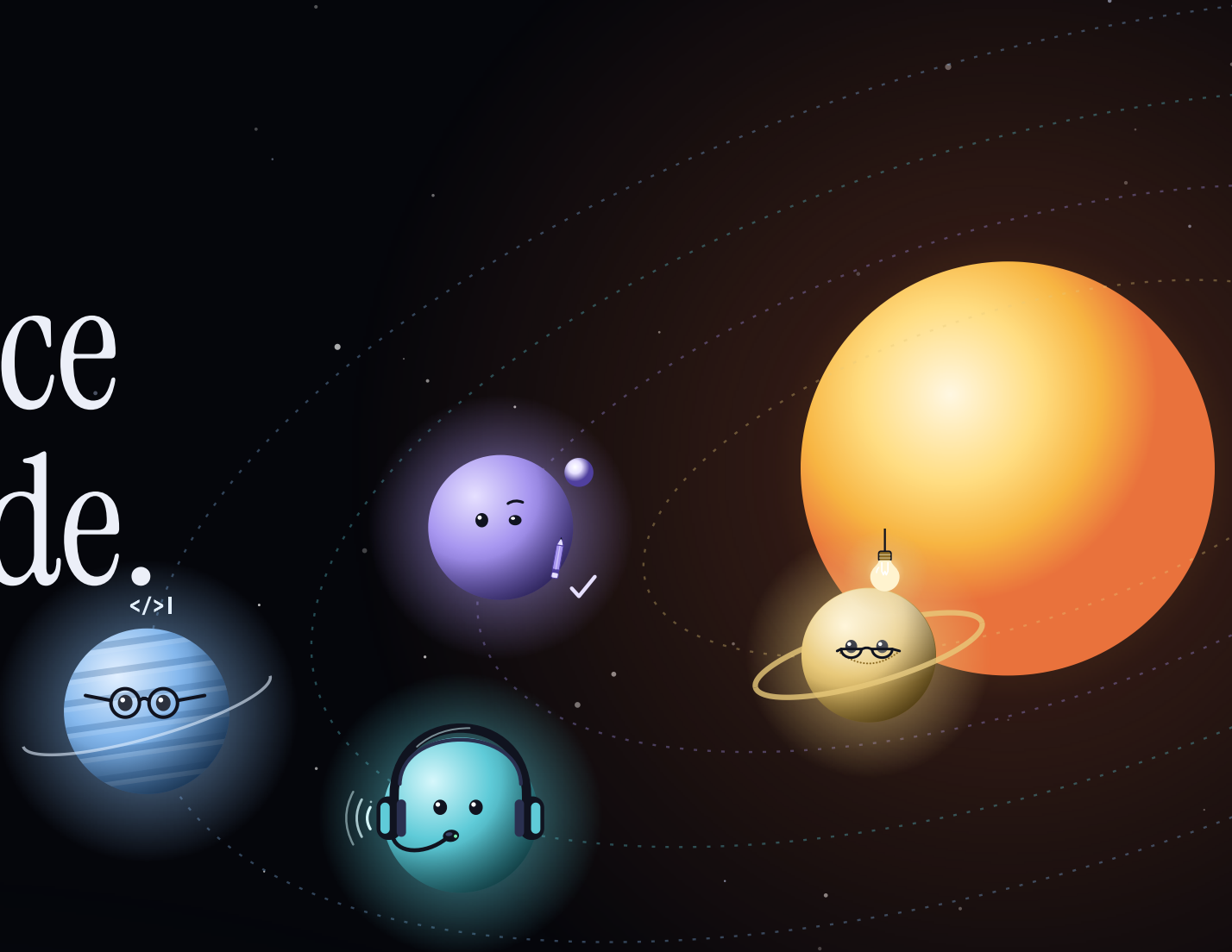


NOX

From a business sentence to *verified, shipped* code.

An agentic platform that keeps a living map of the enterprise, and carries every change through the people who asked for it, and back.



● Gemini

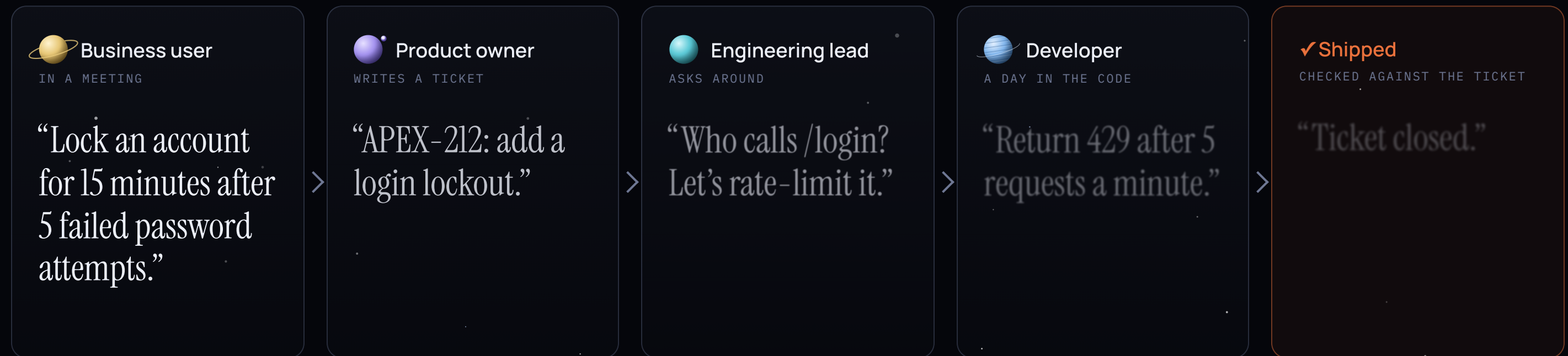
● Agent Development Kit

● Gemma · NoX Local

● Open Knowledge Format

● Cloud Run

In a large enterprise, a requirement travels like a game of *telephone*.



WHAT THE BUSINESS MEANT

WHAT WAS CHECKED AT THE END

Every handoff rebuilds the context from scratch, and loses a little of it. Nobody checks the result against the original sentence.

Everyone already uses AI. *Nobody shares the context.*



Business user

Turn my email into a requirement.

"Here's a polished paragraph..."



Product owner

Write acceptance criteria for this.

"Given... when... then..."



Engineering lead

Which systems does this touch?

"I don't have access to your estate."



Developer

What did the business actually want?

"Could you paste the ticket?"



Context is rebuilt at every seat

Four private chats, four partial pictures, no shared source of truth.



Knowledge goes stale

Wikis drift from the code within a quarter. Agents read outdated maps.



Nobody verifies the intent

The ticket gets checked. The business user's sentence doesn't.

Two halves that make each other better.

Atlas

THE MAP

A live, cited code wiki for every application, in Google's Open Knowledge Format.

- Organizations → teams → applications
- Built from code, Confluence, Jira, Notion, Slack
- Re-checked on every push
- One contract map across the estate



Missions

THE JOURNEY

One change, carried from a business sentence to merged code, and verified back by the same people.

- Four seats, four spec files
- NoX co-writes each one, grounded in the map
- Any coding agent builds it with `/nox`
- Verified in reverse, business user last

Every application gets a code wiki that *stays true*.



Knowledge that *any agent* can read, not just ours.

OKF 0.2 BUNDLE · GIT

- ▼ kb-mini-auth-service/
 - index.md okf_version
 - log.md dated changes
 - ▼ architecture/
 - index.md
 - overview.md
 - ▼ interfaces/
 - index.md
 - login-api.md ◀
 - ▶ components/
 - ▶ decisions/
- + kb-org-apex/ rollup

interfaces/login-api.md

```

---
type: Interface Reference
title: Login API
description: How clients sign in and get a session.
tags: [auth, rest]
sources: [src/auth.py, src/main.py]
generated: nox/gemini-3.7-flash
---

## POST /api/v1/auth/login
Verifies the password and returns a token.
Session rules: [[kb:mini-auth-service/session]]
Called by [[kb:market-data-gateway/auth]]
↳ anchored to src/auth.py:42

```

T

Typed pages

Every page declares a type, title, description and tags.

≡

Progressive disclosure

An index in every folder. Agents read a few lines, then open only what they need.

✓

Provenance built in

sources and generated say where each fact came from.

→

NoX's extensions

[[kb:app/page]] links join many bundles into one enterprise map.

WHY IT MATTERS

● Portable: any OKF-aware agent reads it as is

● Small, fast context for Ask and the co-writer

● Conformance checked on every KB pull request

Four seats. Each with its own *view*, *voice* and *character*.



Business user

REQUEST DESK

WRITES

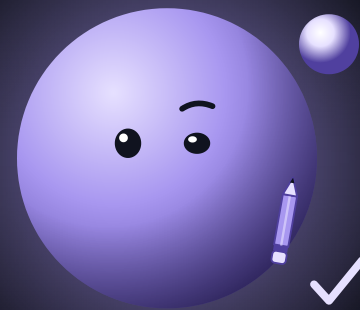
01-business.md · one plain sentence

SPEAKS IN

customers, screens, outcomes

CHECKS AT THE END

"Is my sentence now true?"



Product owner

PRODUCT BOARD

WRITES

02-product.md · goals, criteria, edge cases

SPEAKS IN

users, value, the metric

CHECKS AT THE END

Acceptance criteria and edge cases



Engineering lead

FLIGHT DIRECTOR

WRITES

03-engineering.md · scope, contracts, risks

SPEAKS IN

systems, consumers, failure modes

CHECKS AT THE END

Scope, contracts and guardrails



Developer

BUILD BAY

WRITES

04-developer.md · files, tasks, tests

SPEAKS IN

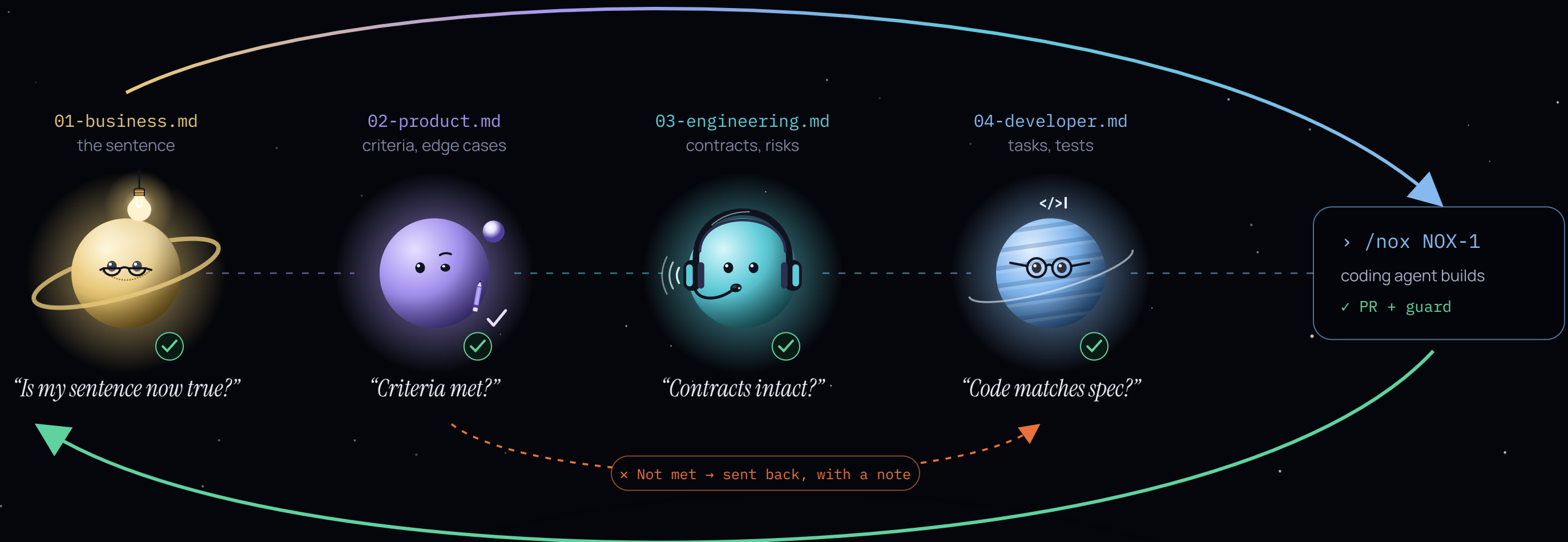
code, reuse, test plans

CHECKS AT THE END

The code against the build spec

Out through the seats. *Back through the same people.*

FORWARD · EACH SEAT WRITES ITS OWN FILE, GROUNDED IN THE ATLAS



REVERSE · EACH SEAT VERIFIES ITS OWN CHECKLIST, BUSINESS USER LAST

SERVER-ENFORCED STAGES

business

→

product

→

engineering

→

developer

→

build

→

verifying

→

done

MIRRORED TO JIRA · COMMITTED TO GIT

THE ONLY THING THE BUSINESS USER TYPED .

“Block brute-force logins: lock an account for 15 minutes after 5 failed password attempts.”

🪐 01-business.md

WHAT CHANGES FOR CUSTOMERS

After five wrong passwords in a row, the account **pauses for 15 minutes**. People who sign in normally notice nothing.

WHY

Stop attackers guessing passwords.

DRAFTED WITH NOX · APPROVED BY BUSINESS USER

☒ My sentence is now true

🌌 02-product.md

ACCEPTANCE CRITERIA

AC-1 Given 5 failed attempts, the 6th is refused for 15 minutes.

EDGE CASE · ADDED BY PO

Lockout must not reveal whether the username exists.

📄 01-business.md

DRAFTED WITH NOX · APPROVED BY PRODUCT OWNER

☒ Every criterion and edge case met

🌊 03-engineering.md

CHANGING

mini-auth-service only

CONTRACTS

POST `/api/v1/auth/login` unchanged. Callers from the contract map.

📄 kb:mini-auth-service/login-api

DRAFTED WITH NOX · APPROVED BY ENGINEERING LEAD

☒ No contract broke

🌐 04-developer.md

FILES

`src/config.py` · `src/auth.py` · `src/main.py`

TASKS

Count failures per account, lock with expiry, same error text for all.

📄 src/auth.py:42

DRAFTED WITH NOX · APPROVED BY DEVELOPER

☒ Code matches the build spec

Same intent, **four vocabularies**. The business user’s file never mentions an endpoint.

Every seat writes with NoX. *Only people approve.*

🔒 01-business

● 02-product · editing

🔒 03-engineering

🔒 04-developer

Acceptance criteria

AC-1 Given 5 failed attempts, the 6th is refused for 15 minutes.

AC-2 A successful sign-in resets the counter.

AC-3 The lock ends on its own after 15 minutes. `kb:mini-auth-service/session` NOX

Success metric

Brute-force sign-ins blocked per day, from baseline to target.

? Open question · Can support unlock an account early?

CHAT WITH NOX

Add a success metric we can measure.

- `search_kb` "login attempts metrics"
- `read_spec_file` 01-business.md
- `replace_section` "Success metric"

Added a metric under **Success metric**. I couldn't find who may unlock early, so I left it as an open question.

↶ Undo turn

Approve · you

📝 Drafts in each reader's own words

💰 Edits one section per tool call

⚙️ Cites every fact it writes

❓ Unknowns become open questions

One command loads the mission into *any coding agent*.

~/mini-auth-service

```
> /nox NOX-1
nox context NOX-1
✓ 4 approved spec files · knowledge-base pages · contracts
✓ Plan follows the Tasks in 04-developer.md
edit src/config.py src/auth.py src/main.py
test all passing
✓ Opened PR "NOX-1: lock accounts after 5 failed attempts"
> nox complete NOX-1
✓ Mission sent back up the chain for verification
> |
```

PULL REQUEST GUARD

NoX comments on the PR: checked against the app's **architecture rules** and linked to its mission.

JIRA MOVES BY ITSELF



EVERYTHING LANDS IN GIT

Spec files commit to `missions/NOX-1-.../` in the app's knowledge-base repo.

WORKS IN

Google Antigravity

Cursor.

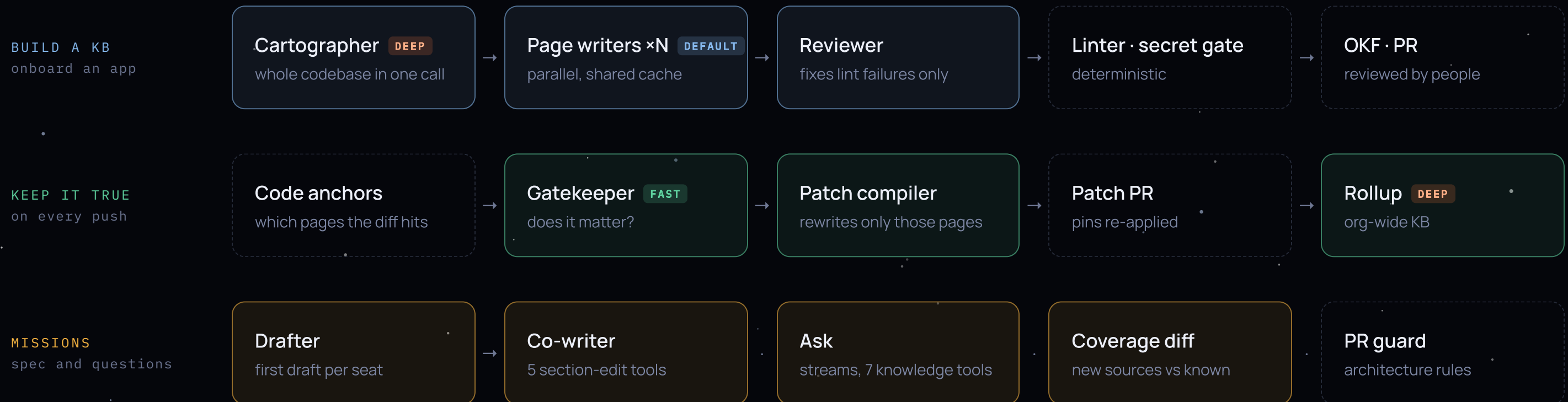
Codex

Copilot

Claude Code

any terminal · nox CLI

Not one prompt. *A team of agents* with real tools.



- Tiers, not model names
- FallbackModel + retries
- Typed output (Pydantic)
- Parallel tool calls
- Long context · ~2.4M chars
- Multimodal diagrams
- Sessions in Postgres
- Streaming steps over SSE
- Per-job token telemetry

The agent team builds a knowledge base $8.7\times$ faster.

29 s

build time, vs 252 s single-model

agents

classic

20k

input tokens, vs 85k (4.25× fewer)

agents

classic

12

model calls, vs 19

agents

classic

Hybrid search

gemini-embedding-2 in pgvector, fused with Postgres full-text. “Refunds” finds a page about “reversals”; exact names still rank first.



NoX Local on Gemma

The same agents on **Gemma 4** via Ollama, on a 16 GB laptop. Code that can’t leave the building still gets a knowledge base.

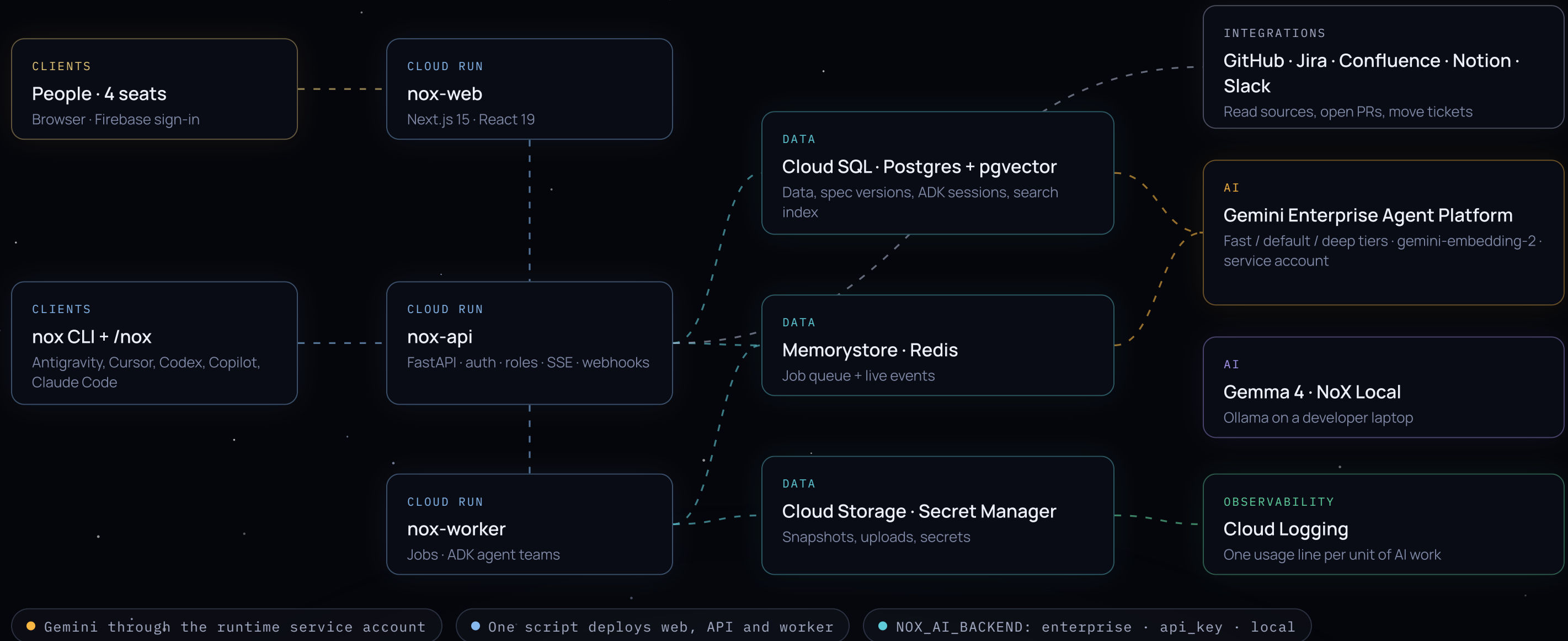
● Implicit caching: first page warms it, the rest reuse it

● 11 focused pages instead of 16 broad ones

● Diagrams read as images

REPRODUCE WITH `SCRIPTS/BENCH_KB.PY` · ASK QUALITY TRACKED BY `SCRIPTS/EVAL_ASK.PY`

Cloud-native, and *no API key* anywhere.



Grounded, scoped, and *always a person's call*.



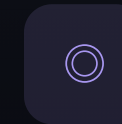
People own decisions

Agents draft, look up and check. Only people approve, verify or send back.



Grounded or it doesn't ship

Every claim cites a KB page, a spec file, a `path:line` or a Jira field.



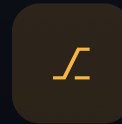
Scope set by NoX, not the model

Tools read access from session state. No prompt can widen what an agent sees.



One human per file

Each spec file has one editing seat. Others read it locked. No merge fights.



Yours, in Git

Knowledge and specs are open-format files in repositories you own and review.



Checked before commit

Linters, secret gate and PR guard stand between model output and your repos.

Less rework. Faster context. *Proof it's what was asked.*



Business user

Asks in one sentence. Signs off against that same sentence.



Product owner

Starts from a grounded draft, not a blank ticket.



Engineering lead

Sees every caller of a contract before approving a design.



Developer

Skips the day of code archaeology. /nox brings the context.

FEASIBLE TODAY

- Working end to end on 5 demo apps in 4 languages
- Plugs into GitHub, Jira, Confluence, Notion, Slack
- No new tool to learn for developers

BUILT TO SCALE

- Cloud Run services scale apart; slow work runs as jobs
- Org → team → app hierarchy with role-based access
- Many KBs, one contract map; Gemma for private code

From one mission to an enterprise that *understands itself*.

Next

Finish the foundations

- Human review recorded in OKF's verified field
- Evidence beside each checklist item
- "Waiting on you" in Slack and Google Chat

10×

Every team, faster

- Onboard a whole GitHub org in an afternoon
- Knowledge tools over MCP, agents over A2A
- Code and BigQuery data in one OKF map

100×

The enterprise as one system

- A living model of all the company's software
- Missions supervised, not driven, for routine change
- An audit trail from requirement to line of code

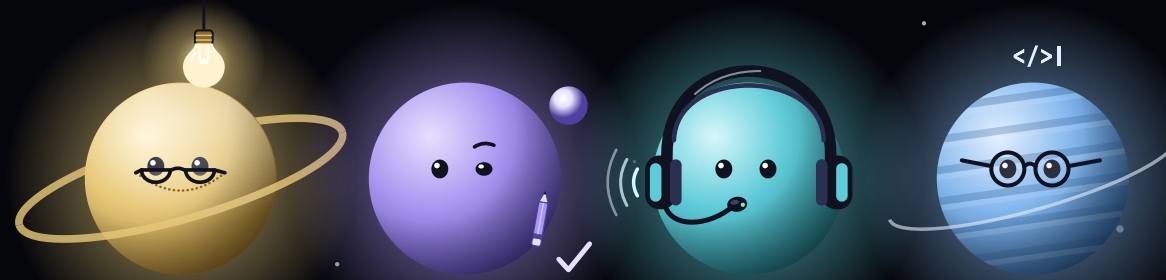
WHAT NEVER CHANGES

● People own decisions

● Everything grounded and cited

● Everything lands in Git

● Each person gets their slice, in their words



Four people. Four files. *One sentence,*
all the way to code and back.

LIVE APP · CLOUD RUN

<deployed URL>

SOURCE · GITHUB

<repository URL>

DEMO VIDEO · 3 MIN

<video URL>



AI BUILDER CUP 2026 · FUTURE OF WORK & ENTERPRISE PRODUCTIVITY